WEEKLY BOX OFFICE

REVENUE FORECASTING

ZAKIYE AMIN 810102398

ROSHAK MASOUDI NEJAD 810102559

PARSA TAHERKHANI 810102579

Mentor : Matin Bazrafshan

Project Overview

Phase 1 - Data Collection & Visualization

In this phase, we gathered data from Box Office Mojo, and used various visualization to understand patterns and trends in the data.

Phase 2 - Data Storage & Preprocessing Pipeline

We designed and implemented a data storage system using structured formats, and built a preprocessing pipeline to clean and transform the data in preparation for modeling.

Phase 3 - Modeling & Prediction

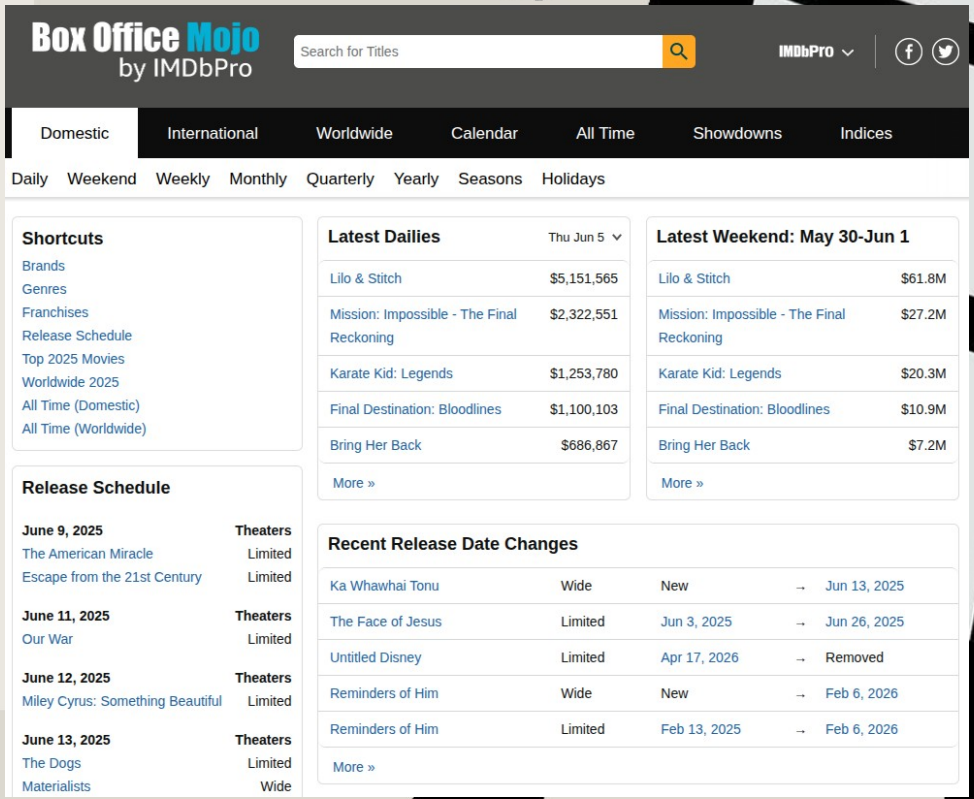
In the final phase, we built machine learning models to predict weekly box office revenue. We evaluated model performance and interpreted the results for real-world insights.

What is Box Office Mojo?

Box Office Mojo is a popular online platform that tracks box office revenue for movies released in the United States and around the world.

It provides detailed financial data including weekly earnings, total gross, opening weekend performance, and historical comparisons across different films.

Box Office Mojo allows users to explore performance trends by studio, genre, release date, or individual movie. It is widely used by film industry professionals, analysts, and movie enthusiasts to monitor the commercial success of films and analyze market trends.



WHY BOX OFFICE MOJO?

Rich datest

Offers comprehensive historical data on box office performance for thousands of movies.

Structured and Clean Data

The data is well-organized and ideal for time-series analysis, forecasting, and machine learning applications.

Relevant Features

Provides important movie metadata such as release date, genre, distributor, runtime, and more—crucial for building predictive models.

Industry Credibility

A trusted and authoritative source in the film industry, widely used by professionals, analysts, and journalists.

Real-World Applicability

Enables practical use cases such as revenue prediction, trend analysis, and box office insights—relevant to producers, studios, and investors.

Focused Domain

Unlike general datasets, Box Office Mojo focuses specifically on the film industry, making it a perfect fit for a project centered on movie revenues.

Why Predict Weekly Box Office Revenue?

- Financial Planning for Studios & Distributors

Accurate forecasts help studios decide on marketing budgets, screen allocation, and release strategies.

- Investment Decisions

Investors and producers can assess potential returns on investment before a movie’s release.

- Resource Allocation

Theaters can optimize screen scheduling and staffing based on expected demand.

- Marketing Optimization

Real-time predictions allow dynamic adjustment of promotional campaigns based on performance trends.

Why Predict Weekly Box Office Revenue?

- Competitive Analysis

Helps understand how a movie might perform relative to others released at the same time.

- Strategic Release Timing

Helps studios select the best week to launch their movies, avoiding clashes with major competitors or holidays.

- Consumer Behavior Insights

Reveals audience preferences over time, useful for shaping future productions and targeting the right demographics.

- Trend Forecasting

Enables industry players to spot long-term patterns and make informed strategic decisions.

Phase 1

Data Collection & Integration

- Scrape Movie-level data
- Scrape Weekly revenue data

Exploratory Data Analysis & Visualization

Data Collection & Integration

Scrape Movie-level data :

```python
driver_path = r"D:\Data Science\Final project\chromedriver-win64\chromedriver.exe"
service = Service(driver_path)
options = webdriver.ChromeOptions()
options.add_argument("--start-maximized")
driver = webdriver.Chrome(service=service, options=options)
url = "https://www.boxofficemojo.com/year/2018/"
driver.get(url)
wait = WebDriverWait(driver, 10)

try:
    cookie_accept_button = wait.until(EC.element_to_be_clickable((By.XPATH, "//button[contains(text(), 'Accept')]")))
    cookie_accept_button.click()
    print("Cookies accepted")
except:
    print("No cookie popup found, proceeding...")
time.sleep(3)
data = []
movie_links = driver.find_elements(By.CSS_SELECTOR, "td a.a-link-normal")
movie_links = [l.get_attribute("href") for link in movie_links if link.get_attribute("href") is not None]
for i in range(min(200, len(movie_links))):
    movie_url = movie_links[i]
    driver.get(movie_url)
    try:
        wait.until(EC.presence_of_element_located((By.TAG_NAME, "h1")))
        title = driver.find_element(By.TAG_NAME, "h1").text.strip()
        def extract_text(xpath):
            try:
                element = driver.find_element(By.XPATH, xpath)
                return element.text.strip()
            except:
                return "N/A"
        try:
            domestic = driver.find_element(By.XPATH, "//span[@class='money']").text.strip()
        except:
            domestic = "N/A"
        try:
            international = driver.find_element(By.XPATH, "//span[@class='money']/parent::a").text.strip()
        except:
            international = "N/A"
        genres = extract_text("//span[contains(text(), 'Genres')]/following-sibling::span")
        release_date = extract_text("//span[contains(text(), 'Release Date')]/following-sibling::span")
        mpaa = extract_text("//span[contains(text(), 'MPAA')]/following-sibling::span")
        running_time = extract_text("//span[contains(text(), 'Running Time')]/following-sibling::span")
        budget = extract_text("//span[contains(text(), 'Budget')]/following-sibling::span")
        in_release = extract_text("//span[contains(text(), 'In Release')]/following-sibling::span")
        if "Become an IMDbPro member today" in title:
            continue
        movie_info = {
            "Title": title,
            "Domestic": domestic,
            "International": international,
            "Genre": genres,
            "Release Date": release_date,
            "MPAA": mpaa,
            "Running Time": running_time,
            "Budget": budget,
            "In Release": in_release}
        data.append(movie_info)

    except Exception as e:
        print(f"Error scraping {movie_url}: {e}")
        continue

movies_df = pd.DataFrame(data)
```

Scrape Weekly revenue data :

```python
driver_path = r"D:\Data Science\Final project\chromedriver-win64\chromedriver.exe"
service = Service(driver_path)
options = webdriver.ChromeOptions()
options.add_argument("--start-maximized")
driver = webdriver.Chrome(service=service, options=options)
data = []

for year in range(1976, 2024, -1):
    print(f"\nScraping year: {year}")
    url = f"https://www.boxofficemojo.com/year/{year}/"
    driver.get(url)
    wait = WebDriverWait(driver, 10)
    try:
        cookie_accept_button = wait.until(EC.element_to_be_clickable((By.XPATH, "//button[contains(text(), 'Accept')]")))
        cookie_accept_button.click()
    except:
        print("No cookie popup found, continuing...")

    movie_links = driver.find_elements(By.CSS_SELECTOR, "td a.a-link-normal")
    movie_links = [link.get_attribute("href") for link in movie_links if link.get_attribute("href")]

    for movie_url in movie_links[:500]:
        driver.get(movie_url)
        try:
            wait.until(EC.presence_of_element_located((By.TAG_NAME, "h1")))
            title = driver.find_element(By.TAG_NAME, "h1").text.strip()
            try:
                weekly_tab = driver.find_element(By.XPATH, "//a[contains(text(), 'Domestic Weekly')]")
                weekly_url = weekly_tab.get_attribute("href")
                driver.get(weekly_url)
                wait.until(EC.presence_of_element_located((By.TAG_NAME, "table")))
                rows = driver.find_elements(By.XPATH, "//table[contains(@class, 'mojo-body-table')]/tbody/tr")
                weekly_earnings = []
                for row in rows:
                    try:
                        earnings = row.find_element(By.XPATH, "./td[3]").text.strip()
                        weekly_earnings.append(earnings)
                    except:
                        continue
                week_numbered = {f"Week{i+1}": earning for i, earning in enumerate(weekly_earnings)}
                movie_info = {"Title": title, **week_numbered}
                data.append(movie_info)
            except Exception as e:
                print(f"Error fetching weekly earnings for {title}: {e}")
        except Exception as e:
            continue
    movies_df = pd.DataFrame(data)
    movies_df.to_csv(f"movies_weekly_earnings_{year}.csv", index=False)
    data.clear()
driver.quit()
```

Type of data we have scraped:

Movie Descriptive Data:

Title, Domestic Revenue , International Revenue, Genre, Release Date, MPAA Rating, Running Time, Budget, Weeks in Release.

Weekly Box Office Performance:

Weekly revenue data for each film across its release weeks.

These datasets were merged based on the movie title to create

a new dataset containing both static features and dynamic performance.

Challenge – Weekly Revenue Formatting

During the data collection process, we encountered an inconsistency in the weekly revenue data:
The weekly earnings were labeled by specific dates rather than sequential week numbers (e.g., "2023-01-06" instead of "Week 1").
To resolve this, we standardized the format by converting date-based columns into a consistent "Week 1", "Week 2", ... structure.
This transformation enabled us to perform uniform comparisons and time-series analysis across different movies, regardless of their release dates.

From this

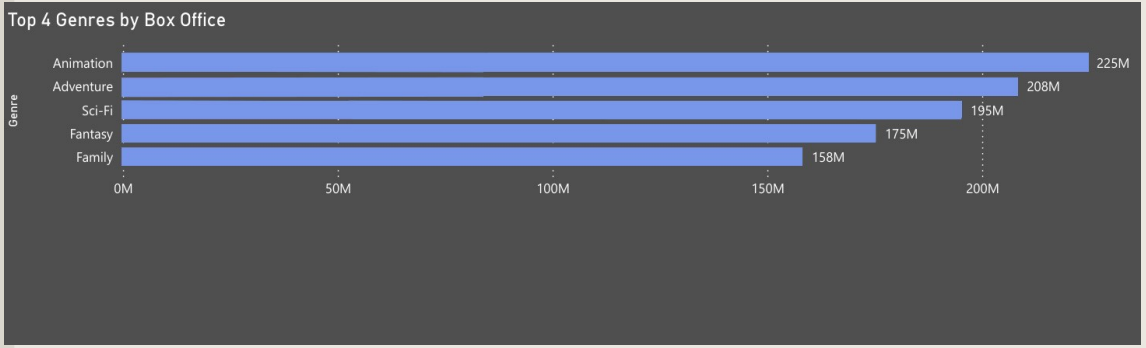
Title	May 20-26	May 27-Jun 2	Jun 17-23	Jun 24-30	Jul 1-7	Jul 8-14
Star Wars: Episode IV - A New Hope	$493,774	$2,062,644	$5,473,681	$9,075,456	$11,228,163	$11,038,368
Looking for Mr. Goodbar	NaN	NaN	NaN	NaN	NaN	NaN
Saturday Night Fever	NaN	NaN	NaN	NaN	NaN	NaN
Close Encounters of the Third Kind	NaN	NaN	NaN	NaN	NaN	NaN
Grease	NaN	NaN	NaN	NaN	NaN	NaN
...	...	...	...	...	...	...
Look Back	NaN	NaN	NaN	NaN	NaN	NaN
Yolo	NaN	NaN	NaN	NaN	NaN	NaN

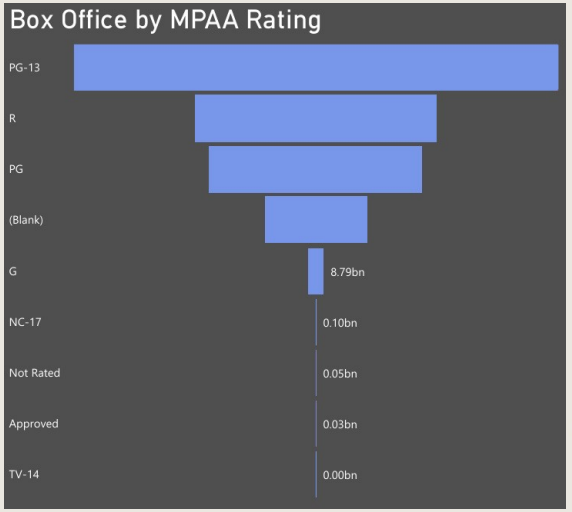
To this

Title	week 1	week 2	week 3	
Star Wars: Episode IV - A New Hope	$493,774	$2,062,644	$5,473,681	$
Looking for Mr. Goodbar	$297,341	NaN	NaN	
Saturday Night Fever	$8,506,454	$7,750,444	$6,922,844	
Close Encounters of the Third Kind	$6,053,118	NaN	NaN	
Grease	$16,055,908	NaN	NaN	
...	...	...	...	
Look Back	$1,308,514	$431,396	$143,387	
Yolo	$1,199,660	$440,902	$178,259	

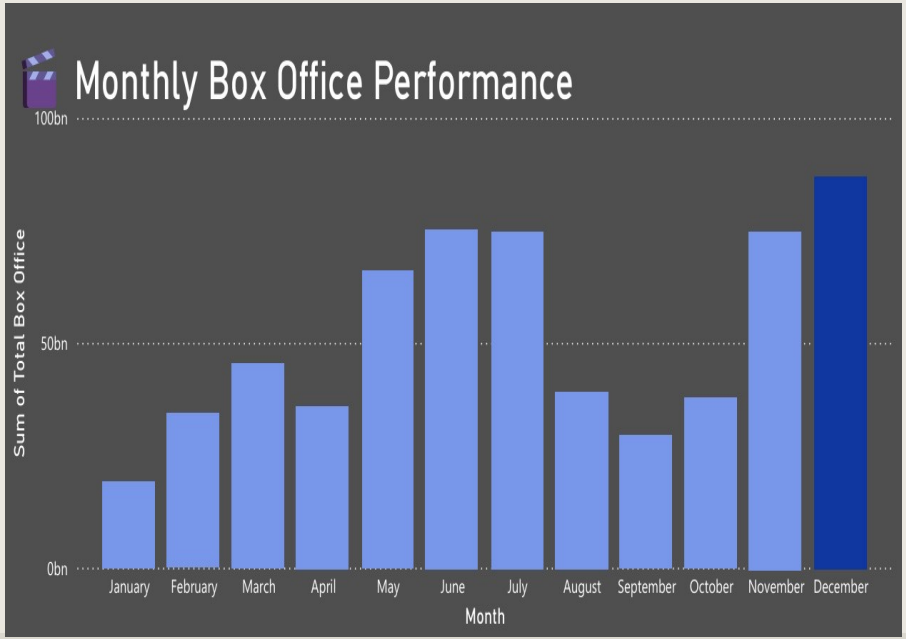
Exploratory Data Analysis & Visualization

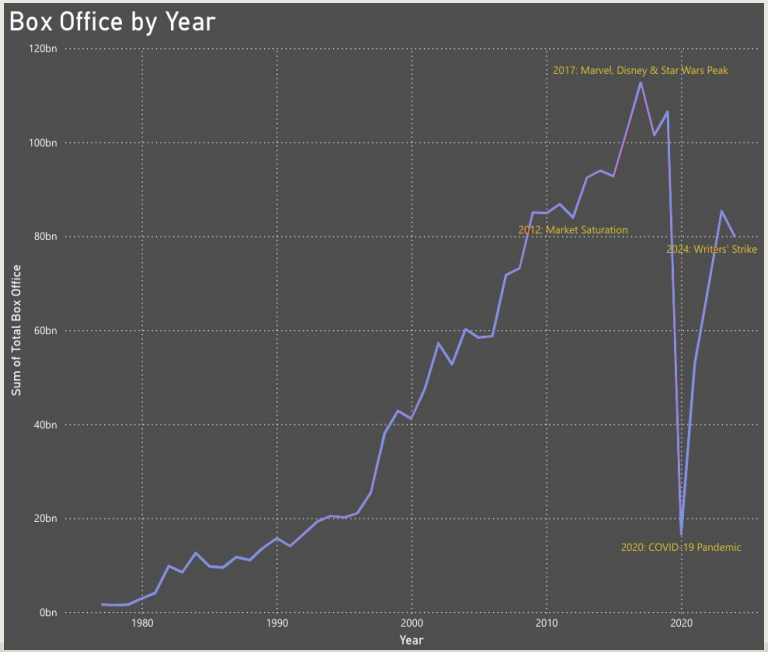
Cinema Gold: Timing, Trends, and Box Office Magic



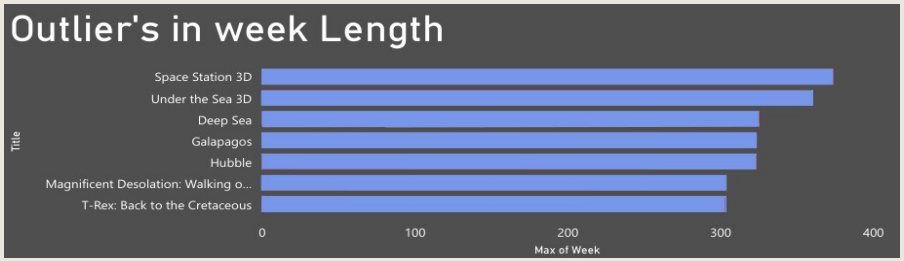


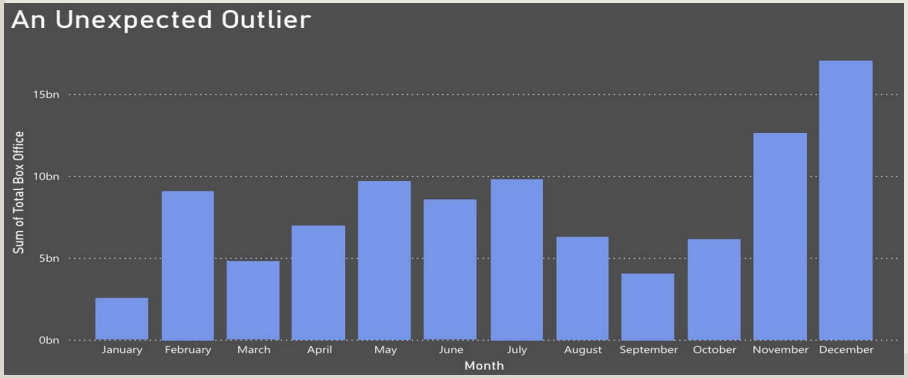
Exploratory Data Analysis & Visualization

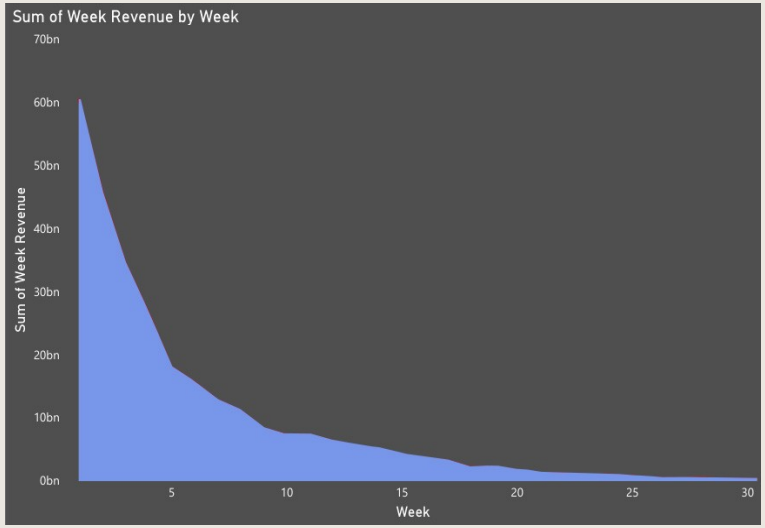




Exploratory Data Analysis & Visualization







Phase 2

Import & Export from database(SQLite)

Data Cleaning

Feature Engineering

Encoding Categorical Variables

Train-Test Splitting

Pipeline Implementation

CI / CID Implementation

Docker, Kubernets

Data Storage & Pipeline Setup

Why Use a Pipeline?

Consistency: Ensures the same sequence of steps is applied to both training and test data.

Reproducibility: Makes it easy to reuse the same process in future runs or new datasets.

Modularity: Organizes code into well-defined, manageable stages.

Efficiency: Reduces human error and speeds up experimentation.

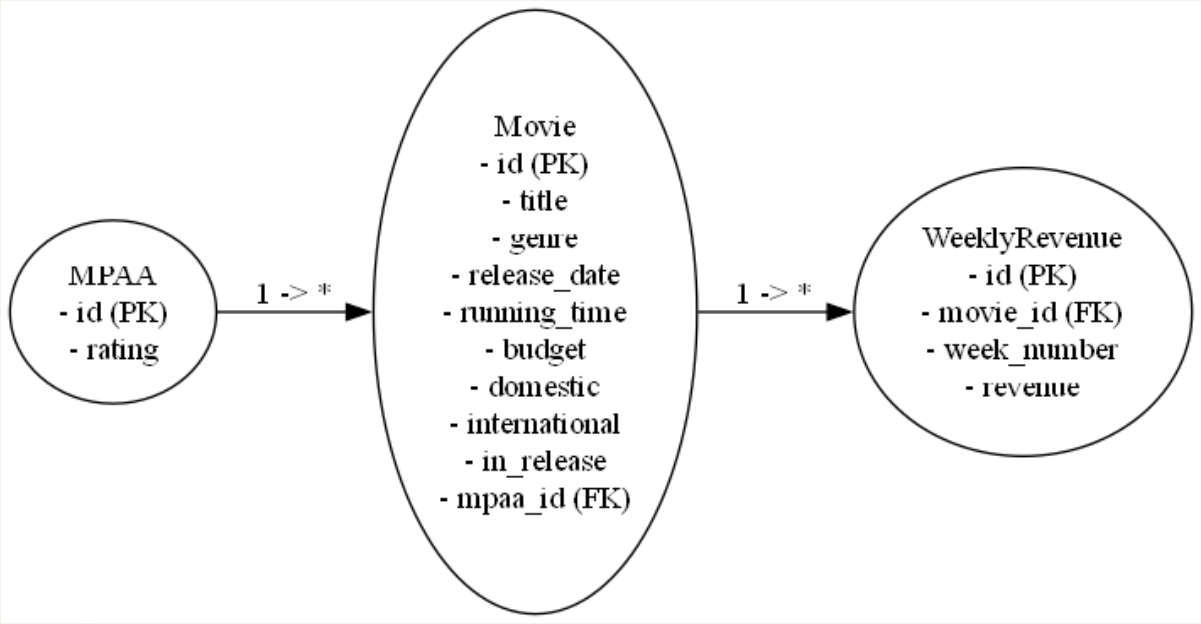
```
df = load_data_train()
df = feature_engineer(df)

X_train, y_train, X_val, y_val = preprocess(df, 'Train')

df = train_model(X_train, y_train, X_val, y_val)
```

```
df = load_data_test()
df = feature_engineer(df)
X_test, y_test, titles = preprocess(df.copy(), 'Test')

predicts = make_prediction(X_test, y_test)

save_predicts = save_predictions(predicts, titles)
```

Pipeline Structure:

Raw Data → Load → Feature Engineering → Preprocessing → Modeling

We first designed 2 databases, one for **train** and one for **test**.



Feature engineering

- Cleaning and Converting Currency Columns

Removed dollar signs ($) and ‘,’ from Domestic, International,

Budget, and week columns.

Converted these string values into integers.

- Extracting Date Components

From the Release Date, extracted:

Month (as a name, then converted to number), Day (as integer), Year

These were added as new columns and the original release date was dropped.

- Converting Runtime to Minutes

Parsed values like 2 hr 30 min from the Running Time column.

Converted total time into a single total_minutes column for simplicity.

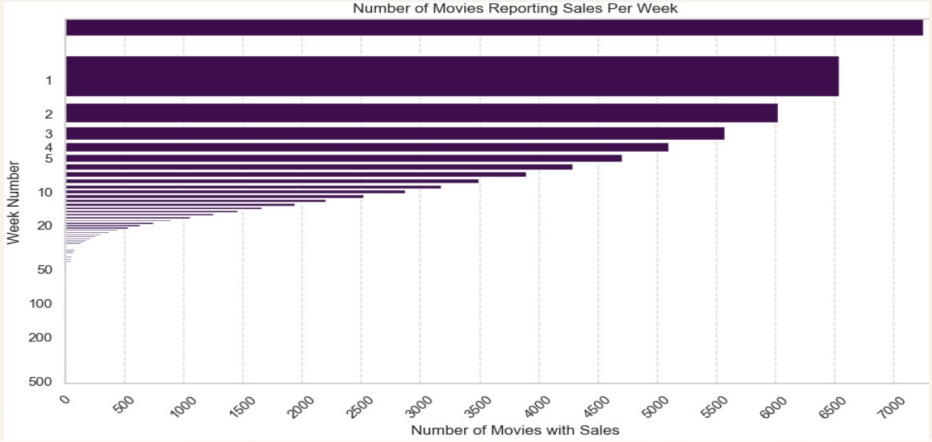
Feature engineering

- Extracting Days in Release

Extracted number of days from strings in the In Release column, Stored as days_in_release to capture the duration a movie was in theaters.

- Selecting First 9 Weeks

Based on data analysis, only the first 9 weeks of box office revenue were kept.



Feature engineering & Encoding Categorical Variables

Melting the Data for Time-Series

Original dataset had one row per movie with multiple week columns. Used pd.melt() to convert it into a long format. Now, each row is equal to a specific week of revenue for a movie. This structure is required for time-series modeling.

Fourier Series for Seasonality

Applied sin/cos transformations on Month and Week using Fourier series.Helps the model understand cyclical patterns. (order = 5)

Encoding Categorical Variables

MPAA Rating: Used one-hot encoding (pd.get_dummies) to convert categorical ratings (like PG, R) into binary columns.

Genre: Genres were multi-label (e.g., "Action Adventure").Used MultiLabelBinarizer to split and binary encode multiple genres per movie.

Feature engineering & Encoding Categorical Variables

New Additions to Feature Engineering

change_from_last_week

Definition: Revenue difference from the previous week.

Purpose: Captures weekly momentum (growth or decline trend).

rolling_mean_2

Definition: Average revenue over the last 2 weeks.

Purpose: Smooths short-term fluctuations to reveal trends.

rolling_std_2

Definition: Revenue volatility over the last 2 weeks.

Purpose: Identifies stability vs. unpredictability in earnings.

total_revenue = Domestic + International and applied log-transform to reduce skew.
Generated time-series features: l***ast_week_revenue*** and ***next_week_revenue*** (as target)

Data Cleaning & Preprocessing

- Manually fixed movies with missing genres (got 40 movies without genre)
- Kept only 10 major genres, dropped the rest.
- Dropped irrelevant columns: Budget, days_in_release, and duplicate genre names.
- Filled missing MPAA ratings using KNN imputation (K=5).
- Encoded Title and Year as numbers, then split data into train, val, test by movie.
- Normalized: total_minutes, Year, and Day using StandardScaler.

  Normalization with Z score, fit and transform on train data and transform on test and validation data.

- Pickle !

Kept only 10 major genres, dropped the rest.

```
Number of movies in Animation genre: 282
Number of movies in Animation genre that have any of the frequent genres: 280
---
Number of movies in Biography genre: 327
Number of movies in Biography genre that have any of the frequent genres: 305
---
Number of movies in Documentary genre: 127
Number of movies in Documentary genre that have any of the frequent genres: 30
---
Number of movies in History genre: 191
Number of movies in History genre that have any of the frequent genres: 177
---
Number of movies in Horror genre: 283
Number of movies in Horror genre that have any of the frequent genres: 245
---
Number of movies in Music genre: 164
Number of movies in Music genre that have any of the frequent genres: 145
---
Number of movies in Musical genre: 104
Number of movies in Musical genre that have any of the frequent genres: 102
---
Number of movies in Mystery genre: 414
Number of movies in Mystery genre that have any of the frequent genres: 393
---
Number of movies in Short genre: 27
...
---
Number of movies in Western genre: 47
Number of movies in Western genre that have any of the frequent genres: 46
```

Phase 3

Baseline Model (Naive Forecast)

Baseline Metrics

Deep Neural Network Model

Training Strategy

Evaluation Metrics

Model vs Baseline Comparison

Baseline

Baseline Model (Naive Forecast)

Our approach is to predict this week's revenue using last week's revenue. It provides a simple benchmark to compare complex models.

Baseline Metrics

MSE = Mean Squared Error

MAPE = Mean Absolute Percentage Error

Deep Neural Network Model

We designed a **4-layer fully connected neural network** to predict weekly box office revenue.

The model takes in all engineered features as input and learns complex, non-linear patterns.

Architecture Overview:

Input Layer: Takes the feature vector for each movie-week.

Dense Layer 1: 128 neurons with ReLU activation – captures high-level patterns.

Dense Layer 2: 64 neurons with ReLU – refines learned features.

Dense Layer 3: 10 neurons with Sigmoid – adds non-linearity and compression.

Dropout Layers: Added between layers to reduce overfitting.

Output Layer: 1 neuron with linear activation – outputs predicted revenue.

Training Strategy

Loss Function: Mean Squared Error (MSE)

Optimizer: Adam (adaptive gradient descent)

Early Stopping: Monitors validation loss to avoid overfitting and restore best weights**.**

Evaluation Metrics

MAPE, and MSE on the test set

Model vs Baseline Comparison

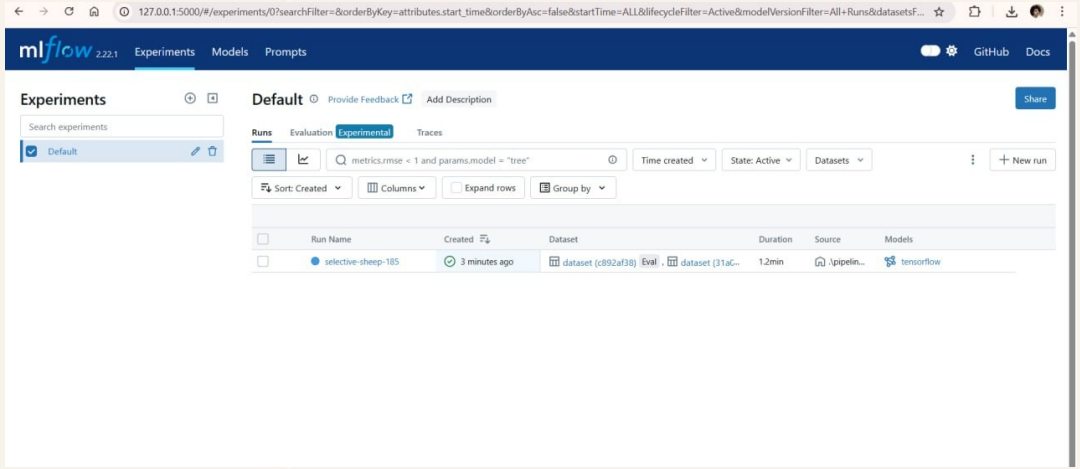
Model accuracy: ✅MSE: 0.1429 MAPE: 2.09%

Baseline accuracy: ✅MSE: 1.2406 Baseline MAPE: 5.40%

Our model is nearly 10 times better in terms of MSE (lower error variance).

And is over 2.5× more accurate in percentage terms.

Phase 3 Bonus : ML Flow



```
🚀 Starting MLflow UI...
✅ MLflow UI is live at: NgrokTunnel: "https://942a-87-98-162-60.ngrok-free.app" -> "http://localhost:5000"
```

✅ Model Trained and Logged to MLflow

Thank you for your patience!